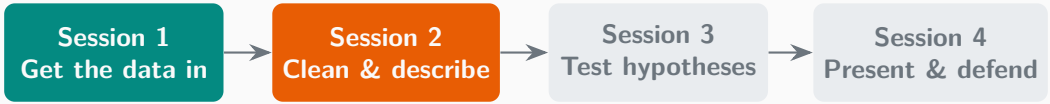
Python for Data Science

Session 2 — Cleaning and Describing the Data

Andrea Guido

PSB Paris School of Business — PGE 5

Last time we got the data in; today we make it trustworthy



Where we left off: 30,000 clients imported, five oddities written down: header row, the missing `PAY_1`, categories stored as numbers, undocumented codes, negative bills.

Today every oddity gets a **decision**, every decision gets a **reason**, and then the data start to talk.

Clients who paid on time in September

17%

Clients two months late in September

69%

Share who defaulted in October. By the end of today you will be able to produce this comparison yourself.

Today: from a raw file to a cleaned dataset and the first patterns

1. **Cleaning is a sequence of decisions**: missing values, duplicates, codes, types, outliers
2. **New variables**: filtering, recoding, and your Session 1 loop in one line
3. **Describing**: means, dispersion, frequencies, cross-tabulations, `groupby`
4. **Visualising**: the plot that answers the question, and how to read it critically

Before class you watched the 20-minute video on cleaning operations in Pandas. We build on it, we do not repeat it.

Part 1: Cleaning is a sequence of decisions

Each one must be written down and justified

The raw file never changes; the cleaning lives in code and in a log



Why it matters: at the oral exam you will be asked "why did you do this?" The log is your answer.

A cleaning log has four columns

Issue	Decision	Reason	Rows
`EDUCATION` = 0, 5, 6	set to missing	not in the codebook; meaning unknown	345
Duplicate rows	kept	distinct IDs; inactive accounts with zero bills	70
Negative bills	kept	credit balance after overpayment	590

The **Rows** column matters: a decision that affects 14 clients and one that affects 14,000 deserve different amounts of worry.

Pandas finds no missing values, yet 399 are hidden in the data

```
df.isna().sum().sum()
```

```
0
```

Explicit missing

An empty cell. Pandas shows `NaN` and `.isna()` finds it.

Disguised missing

A code that means "unknown": 0, 99, -1, "n/a". Pandas treats it as a real value. Here: 345 in `EDUCATION`, 54 in `MARRIAGE`.

Why it matters: an average education code computed with 0s and 6s inside looks precise and means nothing.

A merge with the codebook exposes every undocumented code

```
codebook = pd.DataFrame({
    "EDUCATION": [1, 2, 3, 4],
    "education": ["graduate school", "university",
                  "high school", "others"]})

check = df.merge(codebook, on="EDUCATION",
                 how="left", indicator=True)
check.groupby("_merge")["EDUCATION"].unique()
```

```
both          [2, 1, 3, 4]
left_only     [5, 6, 0]
```

`how="left"` keeps every client; `indicator=True` labels rows the codebook could not match. This is the same tool you use to **merge two data sources**.

Three defensible options for the unknown codes, three different datasets

Option	What happens	Risk
Set to missing	345 clients drop out of RQ1 only	fine if the codes are random errors
Merge into "others"	"others" grows from 123 to 468	mixes unknowns with a real category
Drop the rows	345 clients leave every analysis	loses their data for RQ2 and RQ3 too

We use **set to missing** as the reference. You may choose differently, but you must say why. In Session 4 we will see whether the choice changes the answer.

`.map()` recodes and creates the missing values in one step

```
labels = {1: "graduate school", 2: "university",
          3: "high school", 4: "others"}
df["education"] = df["EDUCATION"].map(labels)   # 0, 5, 6 -> NaN
df["education"].isna().sum()
```

```
345
```

- A dictionary from Session 1, now doing real work: key = old code, value = label
- Any code not in the dictionary becomes `NaN`: now `.isna()` can see it
- The labelled column is a **category**, so we tell Pandas: `.astype("category")`

The 35 duplicate pairs are probably real clients, so we keep them

```
dups = df[df.drop(columns="ID").duplicated(keep=False)]
dups[["LIMIT_BAL", "AGE", "PAY_0", "BILL_AMT1", "PAY_AMT1"]].head(4)
```

	LIMIT_BAL	AGE	PAY_0	BILL_AMT1	PAY_AMT1
18	360000	49	1	0	0
627	230000	39	-1	660	660
839	500000	43	1	0	0
1093	360000	41	1	0	0

Different IDs, mostly inactive accounts with zero bills. Two clients can easily look identical on 24 columns. **Decision:** keep, and log it. (If the IDs were also identical, we would drop them.)

Rename once, at the start, so every later line is readable

```
df = df.rename(columns={"PAY_0": "PAY_1",
                        "default payment next month": "default",
                        "LIMIT_BAL": "limit", "AGE": "age"})

# six columns at once, with a loop inside a dictionary
new_names = {f"PAY_{k}": f"pay_{k}" for k in range(1, 7)}
df = df.rename(columns=new_names)
```

- `pay_1` to `pay_6` now match `bill_1` to `bill_6`: 1 = September, 6 = April
- `f"PAY_{k}"` builds the text `"PAY_1"`, `"PAY_2"`, ... from the number `k`

An outlier is removed only if it is impossible, not because it is inconvenient

Value	Possible?	Likely meaning	Decision
Bill of −165,580 NT$	yes	client overpaid: credit balance	keep
Limit of 1,000,000 NT$	yes	a very good client	keep
Payment of 873,552 NT$	yes	paid off a large balance	keep
Age of 79	yes	a retired client	keep
Age of 250 (hypothetical)	no	typing error	set to missing

Why it matters: dropping large values because they "distort the mean" changes the population you describe. Use the median instead.

Part 2: Creating variables

Filtering, recoding, and your Session 1 code grown up

Filtering keeps the rows that satisfy a condition

```
young = df[df["age"] < 30]                    # one condition
# two conditions, both must hold (& = and)
risky = df[(df["pay_1"] >= 2) & (df["limit"] < 50_000)]
len(young), len(risky)
```

```
(9618, 853)
```

- Inside the brackets is a column of `True`/`False`, one per client
- Combine conditions with `&` (and), `|` (or), and put each one in parentheses

Your Session 1 loop, now for 30,000 clients in one line

Session 1: one client

```
late_months = 0
for status in pay_status:
    if status >= 1:
        late_months += 1
```

Session 2: every client

```
cols = [f"pay_{k}" for k in range(1, 7)]
late = df[cols] >= 1
df["late_months"] = late.sum(axis=1)
```

`df[cols] >= 1` gives a table of `True`/`False`; `.sum(axis=1)` counts the `True`s **across** each row. Same logic, no loop.

Why it matters: this variable will be the star of RQ3. Keep an eye on it.

Two more variables: utilisation and age groups

```
# the Session 1 function, applied to every client at once
df["utilisation"] = df["bill_1"] / df["limit"]

df["age_group"] = pd.cut(df["age"], bins=[20, 30, 40, 50, 80],
                         labels=["21-30", "31-40", "41-50", "51+"])
df["age_group"].value_counts().sort_index()
```

```
21-30    11013
31-40    10713
41-50     6005
51+       2269
```

`pd.cut` turns a number into ordered bins. Check the edges: is a 30-year-old in "21-30" or "31-40"?

Save the clean file: Session 3 starts from it, not from the raw data

```
df.to_csv("data/clean/clients_clean.csv", index=False)
```

The pipeline

Raw file → cleaning notebook → clean file → analysis notebook. Each step can be re-run from scratch.

One catch

A CSV stores text and numbers only. Category types and their order must be set again after loading.

Part 3: Describing the data

Numbers first, then pictures

When the mean sits far above the median, the distribution is skewed

```
df[["limit", "age", "utilisation"]].describe().round(2)
```

	limit	age	utilisation
mean	167484.32	35.49	0.42
std	129747.66	9.22	0.41
50%	140000.00	34.00	0.31
max	1000000.00	79.00	6.46

- **Limit**: mean 167k, median 140k. A few very large limits pull the mean up
- **Dispersion**: the standard deviation (130k) is almost as large as the mean
- **Utilisation** of 6.46: a bill six times the limit. Possible? Log it.

Frequencies: `normalize=True` turns counts into shares

```
df["education"].value_counts(normalize=True, dropna=False).round(3)
```

```
university          0.468
graduate school     0.353
high school         0.164
NaN                 0.012
others              0.004
```

`dropna=False` keeps the missing values visible. Without it, the shares add up to 100% of a smaller sample and you do not see it.

`groupby` is the most useful line in this course

```
df.groupby("education")["default"].agg(["mean", "size"])
```

	mean	size
graduate school	0.192	10585
university	0.237	14030
high school	0.252	4917
others	0.057	123

`groupby("education")`	split the clients by education
`["default"]`	look only at the default column
`.agg(["mean", "size"])`	in each group: the share who defaulted, and how many clients

A cross-tabulation answers the same question as a table of shares

```
pd.crosstab(df["education"], df["default"],
            normalize="index").round(3)
```

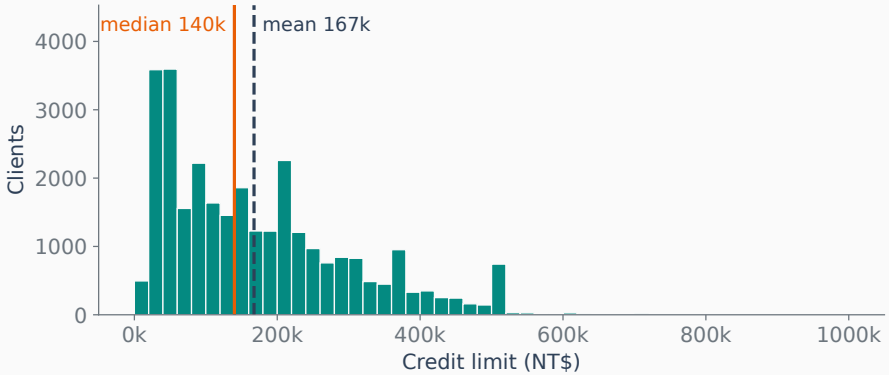
```
default                0      1
education
graduate school    0.808  0.192
university         0.763  0.237
high school        0.748  0.252
others             0.943  0.057
```

`normalize="index"`: each **row** sums to 1. In Session 3 the same table, in counts, feeds the chi-square test for RQ1.

Part 4: Visualising

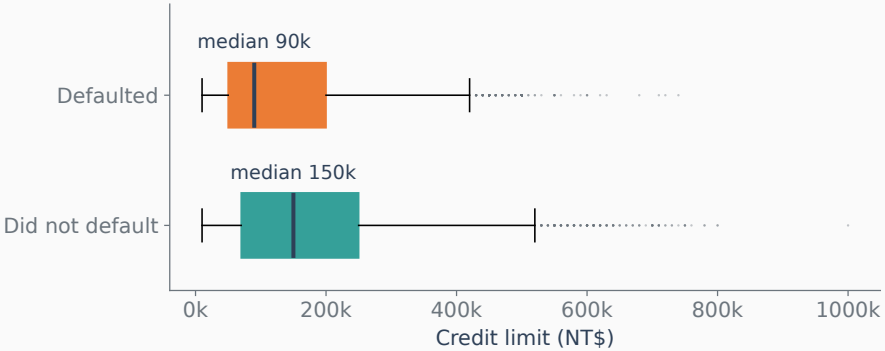
Choose the plot that answers the question

Credit limits are skewed: most clients sit well below the mean



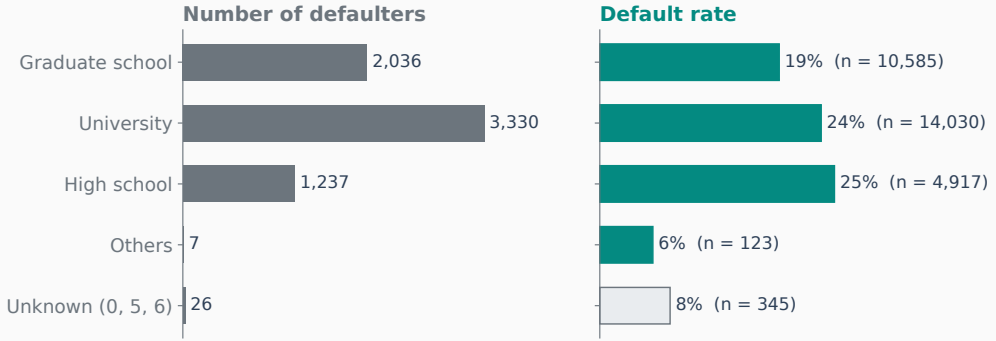
Histogram: the shape of one numeric variable. Notice the spikes at round numbers (50k, 200k, 500k): banks set limits in steps.

Defaulters had lower limits: median 90k against 150k



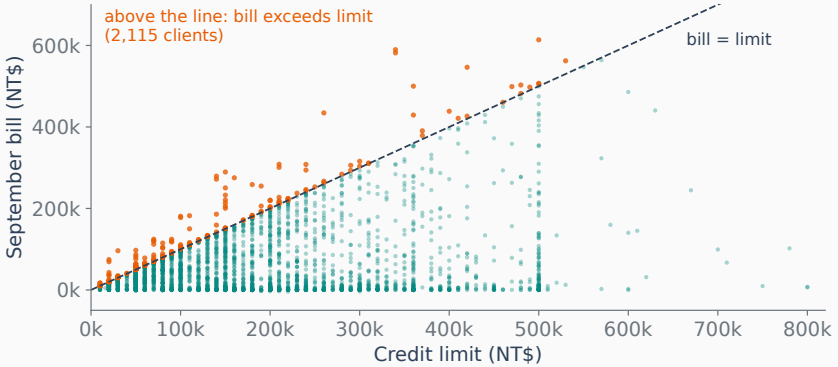
Boxplot: a number compared across groups. The box holds the middle 50% of clients; the line is the median. This is RQ2 in one picture. Is the gap real? Session 3.

Counting defaulters points to university graduates; the rates tell another story



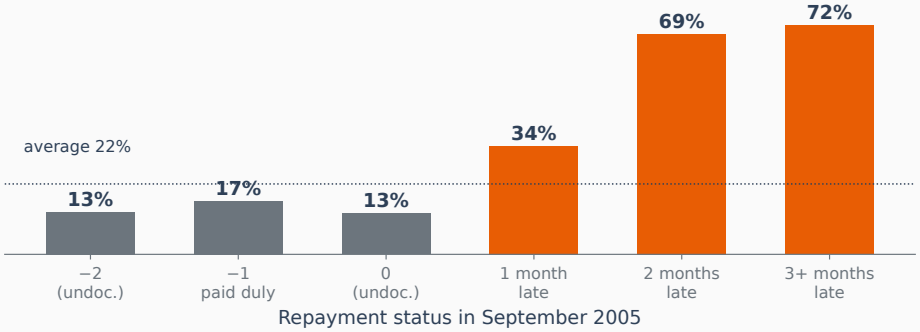
University has the most defaulters because it has the most clients. For RQ1, compare **rates**. And look at the bottom two bars: are 123 and 345 clients enough to say anything?

2,115 clients owed more than their credit limit in September



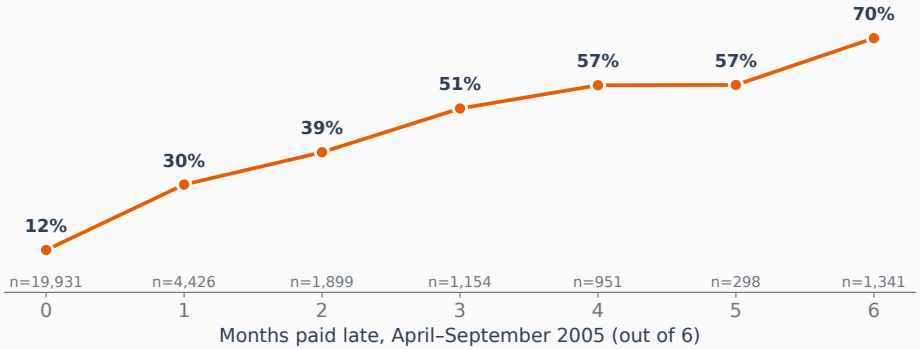
Scatter plot: two numeric variables. A random sample of 4,000 clients is shown, so the points do not hide each other.

Clients two months late in September defaulted 69% of the time



Codes 3 to 8 are grouped (few clients each). Codes -2 and 0 behave like "on time". That is a clue about what they mean.

Each extra late month raises the default rate: 12% to 70%



The variable you built from the Session 1 loop. This is the strongest pattern in the data, and the heart of RQ3.

Match the plot to the question, not to what looks impressive

Your question is about...	Plot	Seaborn
the shape of one number	histogram	`sns.histplot`
a number across groups	boxplot	`sns.boxplot`
a rate or share across groups	bar chart of rates	`sns.barplot`
two numbers together	scatter plot	`sns.scatterplot`

Avoid pie charts and 3D effects: people compare lengths far more accurately than angles or volumes.

Seaborn draws a bar of rates in one line, and adds something new

```
import seaborn as sns
import matplotlib.pyplot as plt

ax = sns.barplot(data=df, x="late_months", y="default")
ax.set(xlabel="Months paid late (out of 6)", ylabel="Default rate")
plt.show()
```

`y="default"` is 0/1, so the height of each bar is the **default rate** in that group.

Each bar has a small black line on top. What is it? **Session 3** answers this.

Before you trust a plot, ask three questions

1

What exactly is on each axis?

Counts or rates? Which month? Which unit?

2

How many observations sit behind each bar?

“Others: 6%” rests on 123 clients.

3

Would a different cleaning decision change it?

Merge the 345 unknowns into “others” and that bar moves.

In-class exercise (feeds your group presentation)

1. **Clean** the raw data in code: names, codes, types, duplicates, outliers. Save it to `data/clean/`.
2. **Log** every decision in a table: issue, decision, reason, rows affected.
3. **Describe** the variables of **one** research question: summary statistics and one `groupby`.
4. **Plot** the pattern that matters for that question, with a title that states what the plot shows.

Work in `session2.ipynb`. The groups choose their own cleaning decisions: justify them, do not copy the reference.

Where we stand: three patterns, none of them tested yet

	What the data show	Open question
RQ1	default rates from 19% to 25% by education	real, or chance?
RQ2	defaulters' median limit: 90k vs 150k	cause, or the bank's choice?
RQ3	12% to 70% as late months rise	does it survive other variables?

Every cleaning decision is a claim you must be able to defend.

Next session: are these differences real, or could they be chance?